

重大 AI 更新提醒

08-31 | llama.cpp 性能优化：KV 缓存恢复提速

本期导读

本期聚焦 llama.cpp 最新 release b10724，针对 KV 缓存恢复进行了关键优化，显著提升长上下文场景下的性能。该更新对使用 CUDA 后端处理大规模提示词缓存的开发者尤为重要。

已检查来源

GitHub Release

1. llama.cpp b10724: KV 缓存恢复优化，非连续单元批量读取

来源 GitHub Release | 类型 GitHub Release | 主题 多模态理解 时间 08-31 12:26 | 分数 7

是什么

这是 llama.cpp 的一个新版本（b10724），主要改进了 KV 缓存（键值缓存）在恢复非连续内存单元时的性能。KV 缓存是 Transformer 模型推理时存储历史键值对的内存结构，用于加速生成。在恢复提示词缓存（prompt-cache）到碎片化环形缓冲区时，旧代码会为每个 KV 单元执行一次小规模复制，导致大量开销。新版本通过预计算连续运行（contiguous runs），将多个连续目标索引合并为一次批量复制，从而大幅减少复制次数。

通俗解释

想象一下，你有一本厚厚的书（模型的历史信息），需要把某些章节（KV 单元）从草稿本（快照）抄写到正式笔记本（缓存）中。旧方法是一页一页地抄，即使这些页在正式笔记本中是连续的，也要逐页处理，非常耗时。新方法则是先找出哪些页在正式笔记本中是连在一起的，然后一次性抄写整段，就像把连续几页一起复印一样。这样，对于 40k+ token 的恢复，复制次数从约 140 万次减少到几十次，在 CUDA 后端上耗时从 25-63 秒大幅下降。

主要作用

解决长上下文场景下 KV 缓存恢复的性能瓶颈，特别是使用 CUDA 后端时，恢复大型提示词缓存（如 RAG 应用中的文档缓存）的速度慢问题。该优化对需要频繁加载和保存模型状态的场景（如多轮对话、服务端推理）有显著价值。

核心功能

批量状态恢复：将非连续目标单元按连续运行分组，每个运行一次复制操作。；预计算运行：在恢复前预先计算连续索引段，并在三个循环（K、V、转置 V）中复用。；字节游标安全：当读写分块不同时，设备端读取器使用字节游标，确保批量读取安全。；字节级一致性：输出与旧版本完全一致（Byte-identical），不影响模型精度。

对比判断

暂无可信公开对比数据，但根据提交信息，在 CUDA 后端上，40k+ token 的恢复时间从 25-63 秒降至更低（具体数值未公开）。

适合谁

适合使用 llama.cpp 进行本地推理的开发者，尤其是依赖 CUDA GPU、处理长上下文（如文档问答、长对话）或使用提示词缓存（如 RAG 系统）的团队。也适合对推理性能优化感兴趣的 AI 工程师。

直达链接 <https://github.com/ggml-org/llama.cpp/releases/tag/b10724>

本简报基于 GitHub Release 信息，仅供参考。